

Accessible Knowledge Management Portal - Code Explanation

This document provides a detailed explanation of the folders, files, and code logic that make up the Accessible Knowledge Management Portal. The project is divided into three main architectures: **Frontend (React)**, **Backend (Spring Boot)**, and **API Gateway (FastAPI)**.

1. API Gateway (FastAPI)

Folder Location: `api-gateway/`

This acts as the single entry point for all frontend requests, securely routing them to the Spring Boot backend.

main.py

- **Line 1-6:** Imports FastAPI, HTTP proxy tools (`httpx`), and CORS middleware to allow the frontend to communicate with the gateway.
 - **Line 8-16:** Initializes the FastAPI app and configures `CORSMiddleware`. This prevents the browser from blocking requests between `localhost:5173` (React) and `localhost:8000` (FastAPI).
 - **Line 18:** Defines `SPRING_BOOT_URL = "http://localhost:8080"`, telling FastAPI where the actual backend database server is located.
 - **Line 20-37 (@app.api_route(...)):** This is a "catch-all" proxy function. Whenever a request hits `http://localhost:8000/api/v1/sql/...`, this function intercepts it, extracts the method (GET, POST, etc.), and forwards the exact same request (along with headers and body) to the Spring Boot server running on port 8080. It then returns the backend's response back to the user.
-

2. Backend Server (Spring Boot / Java)

Folder Location: `backend-sql/`

This handles all business logic, database queries, and Role-Based Access Control (RBAC).

models/ (Database Tables)

- **User.java:** Uses `@Entity` and `@Table(name = "users")`. Defines columns like `id`, `username`, `email`, and `password`. The `@Enumerated(EnumType.STRING)` maps the `role` column to the `Role.java Enum` (ADMIN, MEMBER, USER).
- **Category.java & Document.java:** Defines the `categories` and `documents` tables. They use `@ManyToOne` to establish Foreign Key relationships (e.g., a Document belongs to one Category and one Author).

repository/ (Database Queries)

- **UserRepository.java, CategoryRepository.java, etc.:** These interfaces extend `JpaRepository`. They automatically generate SQL queries (like `SELECT * FROM users`) behind the scenes without requiring manual SQL code.

security/ (Authentication & RBAC)

- **WebSecurityConfig.java:** This is the security firewall. It configures Spring Security to be stateless (using JWT) and defines which URLs are public (like `/api/auth/signin` and Swagger UI) and which require authentication.
- **jwt/JwtUtils.java:** Contains methods to generate JWT tokens upon login, extract usernames from tokens, and validate token signatures.

controllers/ (API Endpoints)

- **AuthController.java:**
 - `@PostMapping("/signin")`: Takes `username` and `password`, authenticates them against the database, and returns a JWT token.
 - `@PostMapping("/signup")`: Reads the requested role, creates a new `User` object, and saves it to the database.
 - `@GetMapping("/users")`: Protected by `@PreAuthorize("hasRole('ADMIN') or hasRole('MEMBER')")`. Only Admins and Members can execute this code to fetch all users.
 - **CategoryController.java & DocumentController.java:** Contains the CRUD (Create, Read, Update, Delete) logic. For example, `@DeleteMapping("/{id}")` takes an ID, searches the repository, and deletes the record if found, returning a 200 OK status.
-

3. Frontend UI (React)

Folder Location: `frontend/`

This is the modern, glassmorphism user interface the client interacts with.

src/App.jsx & src/main.jsx

- **main.jsx:** The React entry point. It injects the App into the HTML DOM.
- **App.jsx:** Uses `react-router-dom` to define the website's URLs. It maps `/login` to the Login component, `/signup` to Signup, and `/dashboard` to the main Dashboard component.

src/pages/ (Main Pages)

- **Signup.jsx:**
 - Uses React `useState` hooks to track the `username`, `email`, `password`, and `role` typed into the form.
 - The `handleSignup` function uses `axios.post` to send this data to the API Gateway. If successful, it triggers a redirect to the login page.
- **Login.jsx:**
 - Takes `username` and `password`, sends them to `/api/v1/auth/signin`.
 - Crucially, it saves the returned JWT token, `username`, `email`, and `role` to the browser's `localStorage` so the app "remembers" who is logged in.
- **Dashboard.jsx:**
 - This is the central hub. It reads the user's `role` from `localStorage`.
 - It conditionally renders different tabs. If the `role` is `ROLE_USER`, the `{activeTab === 'documents'}` logic completely hides the Documents and Users buttons, protecting the UI from unauthorized access.

src/components/ (Dashboard Tabs)

- **Modal.jsx**: A reusable, blurred "glassmorphism" popup container. It accepts `children` (HTML elements) to render different forms dynamically.
- **CategoryManager.jsx & DocumentManager.jsx**:
 - `fetchData()`: Uses `axios.get` with an `Authorization: Bearer <token>` header to securely fetch data from the backend.
 - `handleSubmit()`: Depending on whether the user is creating or editing, it triggers an `axios.post` or `axios.put`.
- **UserManager.jsx**:
 - Contains security checks: `{localStorage.getItem('role') === 'ROLE_ADMIN' && (...)}`. This line ensures that even if a Member views the Users tab, the React code will refuse to render the Edit and Delete HTML buttons.
- **Profile.jsx**: Extracts user data from `localStorage` and dynamically renders an avatar by taking the first letter of the username (`username.charAt(0).toUpperCase()`).

Generated securely by Advanced Agentic System for Project Review.